

版本/分支感知的多状态记忆闭环

TencentDB Agent Memory 实现与真实评测 | 2026-08-31 | 分支 codex/version-aware-multistate-memory | 评分前提交 4d77e69

技术结论：相对原始 Agent Memory 全局混合召回，70 个冻结版本能力题的答案准确率由 94.29% 提升到 100.00%，提升 5.71 点 [2.86, 8.57]；同时精确状态选择率 0% → 100%，跨状态污染率 100% → 0%，注入 token -29.04%，召回条目 -67.86%。7 题改善、63 持平、0 受损。

+5.71 点	0% → 100%	100% → 0%	-29.04%	-67.86%
答案准确率	精确状态选择	跨状态污染	注入 token	召回条目

原始方案 = 同一 performAutoRecall 生产路径中关闭版本感知、直接使用全局 Top-k 的 global_latest；不是手工 mock，也没有更换 Reader/Judge。

1 原理：同域更新，跨域并存

线性“旧→新”只适用于同一有效域。main、release、多个 worktree 和并行 Agent task 可以互相矛盾但同时正确；不同域并列保存，查询时按当前环境或问题意图选择。repo/worktree 为不可逆哈希，不保存远端 URL 或本机路径；commit 用于溯源。



层级	有效域键	适用条件	典型状态
repository	repo	仓库相同	项目级构建约定
branch	repo + branch	再匹配分支	release 测试命令
worktree	repo + branch + wt	再匹配 worktree	未提交配置、实验环境
task	repo + branch + wt + task	再匹配 taskId	并行任务假设、临时命令

为什么不能直接用“新记忆”全局覆盖“旧记忆”

时间新旧不等于事实有效性。例如 main 已升级到 Java 21，而 release/1.x 仍要求 Java 17；main 上较新的写入不能删除 release/1.x 的状态。只有两个事实的有效域键完全相同时，系统才允许 update、merge、delete 或建立纠错边；跨分支、跨 worktree、跨 task 的状态均保留为并列真值。

具体例子：Agent 在 main 提问“当前 Java 版本”时只得到 Java 21，在 release/1.x 提问时只得到 Java 17；提问“比较 main 与 release”时才同时得到两条带分支标签的状态。如果 main 后续升级到 Java 22，系统只在 main 域内更新 21，不影响 release/1.x 的 17。

查询决策：当前态问题只返回执行环境匹配的状态；比较、迁移、回归或历史问题才进入有界多状态模式，并标记 ACTIVE SCOPE / VERSION STATE。缺少仓库上下文时抑制 scoped 候选，只兼容 legacy unscoped，不猜当前值。

系统维护三条不变量：状态身份不可丢失，每条 scoped 记忆都可追溯到 repo/branch/worktree/task；更新只在同域发生，新时间戳不能跨域覆盖事实；返回集合有界，当前态优先最具体有效域，多状态查询也受 maxVersionStates 限制。

选择器对普通问题先过滤当前坐标可用的候选，再按 task → worktree → branch → repository 的具体程度排序；有 scoped 命中时 legacy 不参与竞争。多状态问题限定在同一 repository，先为每个有效域保留一条，再按原检索顺序补齐，因此既保留多样性又不破坏相关性排序。

闭环怎么形成

宿主传入 workspaceDir、稳定 taskId 或显式 versionContext；写入阶段把版本坐标固化进 L1 metadata，同域去重只处理完全相同的有效域。召回先从真实 SQLite/FTS5 检索池按 4 倍过取，再执行生命周期解析与版本选择，避免正确状态被其他分支挤出原始 Top-k。

每次召回记录查询意图、候选数、被抑制条数、返回状态数、fallback、token 和延迟。反馈只用于调整候选倍率、最大状态数和意图规则，不会自动跨域改写事实；状态内容仍由明确写入或同域纠错更新，从机制上阻断反馈污染。

2 TencentDB Agent Memory 接入与 GitHub 代码

Fork 分支: [NianJiuZst/TencentDB-Agent-Memory](#) · [codex/version-aware-multistate-memory](#)

环节	GitHub 代码	接入职责
作用域与选择	version-scope.ts	域键、有效性、意图、标签、多状态选择
Git/worktree	git-context.ts	仓库/worktree 身份、detached HEAD、隐私哈希
并行任务延续	version-context-store.ts	按 session + task 保存坐标
最终生产召回	auto-recall.ts	过取、生命周期解析、版本筛选和提示标签
同域写入	l1-dedup.ts · l1-writer.ts	只在完全相同有效域内更新和建边

接入现有 Agent，不替换原有记忆链路

开启 recall.lifecycle.versionAwareMode = strict 和 autoDetectGit 后，宿主只需把当前工作目录和并行任务标识传给原生产调用。performAutoRecall 优先使用显式 versionContext，否则从 workspaceDir 自动探测 Git；writeMemory 接收同一个 versionContext 并写入 L1 metadata。原有检索策略、生命周期解析、Reader 和提示注入接口保持不变。

```
performAutoRecall({ ...baseRecallArgs, workspaceDir, taskId })
writeMemory({ ...baseWriteArgs, versionContext: detectedContext })
```

生产参数使用候选过取倍率 4、最多 6 个版本状态。普通当前态通常只注入 1 条；只有问题显式要求比较、迁移、回归或历史时才返回多状态。Git 探测失败走安全 fallback，不改变 legacy unscoped 行为。

运行时调用链

- ① 捕获：插件入口按 workspaceDir 探测 repo、branch、commit 和 worktree，并把 taskId 作为并行任务维度；显式坐标优先于自动探测。
- ② 写入：L0 抽取结果继承 session + task 对应的坐标，writeMemory 将其写入 metadata；l1-dedup 只比较同域候选。
- ③ 召回：auto-recall 在现有检索之后执行生命周期解析与版本选择，再把带来源标签的结果写回原有 prependContext。

Git 坐标：repositoryId 优先对规范化 remote 做 SHA-256 短哈希，无 remote 时使用 git common-dir；worktreeId 对真实顶层目录做同样哈希。detached HEAD 写成 detached@提交前缀，同时保存完整 commitSha。探测超时为 250 ms、缓存 2 秒，异常返回 undefined 而不是泄露路径或猜测身份。

3 数据集与实验方法

数据面板	来源与规模	用途与边界
版本能力	10 场景 × 7 类题 = 70 题	验证 branch/worktree/task/比较/迁移/回归/缺失 scope；受控生成
Memora 安全	固定 a6493188...；10 persona × 4 = 40 题	仅验证 legacy 不退化；复用公开面板，不是新留出集
Memora 原始规模	600 问题；27,614 session；24,856 memory unit	activity/preference/goal 的 add/update/delete 与 no-memory

真实生产路径与公平对照

实验冻结原始全局混合、线性新旧双态和版本感知多状态三组对照，110 题各跑 3 组。环境包含真实 Git 仓库的 main、release、两个 detached worktree 和两个并行 task；50 个真实 SQLite/FTS5 数据库共执行 260 次 writeMemory，最终答案上下文全部经过 performAutoRecall，共 330 次召回且 fallback 为 0。

70 个能力题由 10 个独立编程场景分别生成 7 类查询，场景是 bootstrap 聚类单位。40 个 Memora 安全题按固定规则从 10 个 persona 各取 4 题，选择时不查看答案或 verdict；它用于兼容性检查，不冒充新的外部留出集。

答案由 MiniMax-M3 与 deepseek-v4-flash 各读取 250 次，再由另一固定模型逐 criterion 交叉判断，共 500 次 Judge；无自评、错配、重试或 unclear。95% 区间按 10 个场景/persona 做 5,000 次配对聚类 bootstrap；独立验证器从 500 条原始 verdict 重算指标、门槛和哈希，mismatchCount = 0。

公平性：三组使用相同数据、SQLite 写入、检索策略、result limit、提示预算、Reader 与 Judge；唯一变化是版本状态的组织 and 选择策略。因此答案差异可以归因到召回上下文，而不是模型或数据切换。

指标口径：主指标为 criterion accuracy；上下文指标为期望集合精确率、期望状态召回率和污染率，同时记录 token、召回条目与本地耗时。能力题同时检查正确值是否出现、兄弟状态是否被误当成当前值。

反馈证据：每次生产召回保留选择意图、候选与抑制数量、返回状态数、fallback、token 和耗时；答案侧保存 Reader 输出与交叉 Judge verdict。两条证据链通过 caseId 对齐，使“召回了什么”和“最终答对没有”可以独立复算。

4 实验结果

70 题能力面板	原始全局混合	线性新旧双态	版本感知多状态	相对原始提升
criterion accuracy	94.29%	67.50%	100.00%	+5.71 点
MPA / FAA / FAMA	94.29 / 94.29 / 94.29%	47.14 / 87.86 / 47.14%	100 / 100 / 100%	均 +5.71 点
期望集合精确率	0.00%	0.00%	100.00%	+100.00 点
期望状态召回率	85.71%	21.43%	100.00%	+14.29 点
跨状态污染率	100.00%	100.00%	0.00%	-100.00 点
平均注入 token	557.89	487.57	395.90	-29.04%
平均召回条目	4.00	2.03	1.29	-67.86%
本地平均召回	0.622 ms	0.822 ms	0.569 ms	-8.52%

如何解读：+5.71 点是高准确率基线上的答案增益；更大的系统收益体现在上下文结构由“正确答案夹杂冲突状态”变为“正确且纯净的状态集合”。因此精确选择和污染率比单独的答案分数更能说明多分支记忆是否安全。

线性新旧双态为什么下降

线性双态把本应并列的 branch/worktree/task 状态强行解释成一条 old → current 链，结果在非当前域问题中丢失有效状态，criterion accuracy 降到 67.50%。这不是更换模型造成的，而是“时间顺序可以代表全部有效性”的建模假设不适用于并行开发。版本坐标把时间关系降为同域内的更新依据，从根本上消除该冲突。

提升来自状态纯度，而不是牺牲已有能力

原始方案在分支、worktree 和并行任务的“正确值出现”题上本来就能达到 100%，但会同时召回兄弟状态，因此期望集合精确率为 0%、污染率为 100%。版本感知方案保留正确值的同时清除无关状态；比较、迁移和回归问题仍按意图返回多个有标签版本。缺失 scope 时的安全弃权从 60% 提升到 100%，这是答案准确率净增益的主要来源。

稳健性检查没有发现受损样本

相对原始方案逐题统计为 7 改善、63 持平、0 受损；两位 Reader 的准确率变化分别为 +2.86 和 +8.57 点，方向一致。40 题 Memora legacy 安全面板中三组提示和答案完全相同，FAMA 维持 2.19%，只说明旧的 unscoped 路径未退化，不把它解释为通用质量提升。独立验证对 500 条 verdict 的复算全部一致。

实际使用价值

多 worktree 或并行 Agent 同时修改不同版本时，每个任务只看到自己的构建命令、依赖版本和临时假设，避免一个分支的 L0/L1 状态污染另一个分支。处理迁移、回归或发布差异时，系统又能主动取回多个带来源标签的状态供 Agent 对比，不必删除仍然有效的历史事实。token 减少 29.04%、条目减少 67.86%，也降低了模型在冲突上下文中选错状态的机会。

最典型的受益场景是长期维护 main/release、多 worktree 并行开发、多个 Agent 同仓库分工，以及迁移和回归分析。它们共同特点不是存在一个绝对“最新事实”，而是存在多个各自在特定执行坐标上有效的事实。

5 证据边界

已验证：真实 Git/worktree 探测、隐私坐标、并行 task 延续、SQLite 写入、同域去重、最终生产召回、版本选择、标签、双 Reader/交叉 Judge 和 legacy 兼容。

未覆盖：自动 LLM 在自然对话中总能抽取正确 scopeLevel、在线 TCVDB/COS 延迟、复杂 merge/cherry-pick/分支重命名、自然流量冲突率和人类 Judge 标定；因此本地耗时下降不能直接外推为线上数据库性能。

下一轮验证应优先补足三项：从真实开发对话构造自然冲突集，在线后端复测 P50/P95 延迟与 token 预算，并用人工抽检校准 Judge；这些检查不会改变本轮已冻结结果，只用于判断外部泛化和线上成本。

成果长处：在不更换 Reader/Judge、不过度依赖较弱双态基线的前提下，本方案相对原始 Agent Memory 同时取得答案正增益、状态集合完全正确、跨状态污染清零和显著 token/条目节省；这是面向多分支、多 worktree 与并行 Agent 的系统级改进。

证据 context-manifest.json · evaluations.jsonl · summary.json · independent-validation.json | 上下文 2c64c12d...a1960f | 原始评测 e546f4f9...65ec61

代码 [GitHub fork 分支与完整实现](#)